

Axela Brockett UE5 Plugin: Utility AI

v1.1.1 Documentation

GitHub download link: <https://github.com/AxelaBrockett/Utility-AI-Plugin-UE5.6>

Contents:

Basic Overview of the Plugin.....	3
FAxelaBrockettUtilityAIBaseModule.....	4
FAxelaBrockettUtilityAIEditorBaseModule.....	5
UABUtilityComponent.....	6
UABUtilityActionBase.....	15
UABUtilityRequirementBase.....	22
UABUtilityFactorBase.....	24
UABUtilityActionBlueprintBase.....	26
UABUtilityRequirementBlueprintBase.....	29
UABUtilityFactorBlueprintBase.....	31
UABUtilityDataAsset.....	34
UABUtilitySettings.....	35
UABUtilitySubsystem.....	36
FGameplayDebuggerCategory_Utility.....	39
UABUtilityDataAssetValidator.....	40
ABUtilityTypes.h.....	41
ABUtilityHelpers.h.....	42
Examples.....	45
Component Setup.....	45
Code Action.....	46
Blueprint Action.....	47
Action Setup in Data Asset.....	47
Code Requirement.....	47
Blueprint Requirement.....	48
Multi-Requirement.....	48
Code Factor.....	48
Blueprint Factor.....	49
Requirement and Factor Setup in Data Asset.....	49
Asynchronous Logic and Parallel For.....	49
Validation.....	50
Debugging.....	50
Comments.....	51

Basic Overview of the Plugin

Utility AI is a decision making method for AI that gives the AI a list of Actions to choose from, gives each of those Actions a score, and then the AI picks the most appropriate Action, most commonly the highest scoring one. This plugin uses the following terms throughout:

- Action
 - A task the AI can do, similar to a Behavior Tree task or State Tree task
- Requirement
 - A prerequisite that must be passed for the action to be available in the moment
- Factor
 - A Factor is considered when calculating the score for an Action

Other terms will also be explained as they appear within the documentation.

The plugin contains a Utility Component, base classes for an Action, a Requirement and a Factor, a Data Asset class, helper files, debugging and validation code. This documentation will go over each of the available classes and how to use each of them to create a usable AI.

FAxelaBrockettUtilityAIBaseModule

This is the base module class that is used for the runtime part of this plugin.

Parent class: *IModuleInterface*

Functions:

Function	Params	Use/Logic
<i>IModuleInterface implementation</i>		
<i>virtual void StartupModule() override</i>		Runs automatically when the module is loaded into memory. Registers the “UtiltiyAI” gameplay debugger category.
<i>virtual void ShutdownModule() override</i>		Runs automatically when the modules are being cleaned up. Unregisters the “UtilityAI” gameplay debugger category.

FAxelaBrockettUtilityAIEditorBaseModule

This is the base module class that is used for the editor part of this plugin.

Parent class: *IModuleInterface*

Functions:

Function	Params	Use/Logic
<i>IModuleInterface implementation</i>		
<i>virtual void StartupModule() override</i>		Runs automatically when the module is loaded into memory.
<i>virtual void ShutdownModule() override</i>		Runs automatically when the modules are being cleaned up.

UABUtilityComponent

This is the Brain Component that is run by the AI Controller. It contains a soft pointer (unloaded) to a Utility Data Asset that has the available Actions, and has both synchronous and asynchronous logic available to the user. The component is completely event driven, so ticking is actively turned off in the constructor, and the *SetComponentTickEnabled(bool bEnabled)* function has been overridden to ensure tick does not get activated. It is not recommended to allow this component to tick.

Parent class: *UBrainComponent*

Functions:

Function	Params	Use/Logic
<i>UABUtilityComponent()</i>		Constructor. Turns tick off.
<i>void SetComponentTickEnabled(bool bEnabled) override</i>	<i>bEnabled</i> - Usually allows users to activate or deactivate the tick functionality	Ensures Tick does not get activated.
<i>void StartLogic() override</i>		Subscribes to the Actions within the data asset set in <i>UtilityDataAssetClass</i> through the <i>UtilitySubsystem</i> , binds to each Actions delegates for completion and Requirement checking, and attempts to perform an Action.
<i>void PauseLogic(const FString& Reason) override</i>	<i>Reason</i> - Allows users to specify why the component is pausing	Logs the <i>Reason</i> .
<i>EAILogicResuming::Type ResumeLogic(const FString& Reason) override</i>	<i>Reason</i> - Allows users to specify why the component is now resuming	If <i>ShouldRestartLogicWhen ResumingComponentLogic()</i> returns true, runs <i>RestartLogic()</i> . Otherwise, attempts to resume the current Action if there is

		one. If there is not one, attempts to find and run a new Action.
<i>void RestartLogic() override</i>		Runs <i>StopLogic()</i> followed by <i>StartLogic()</i> .
<i>void StopLogic(const FString& Reason) override</i>	<i>Reason</i> - Allows users to specify why the component is stopping	Aborts the current Action if there is one, unbinds from all available Actions delegates for completion and Requirement checking, and unsubscribes from the Actions through the <i>UtilitySubsystem</i> .
<i>void Cleanup() override</i>		Runs <i>StopLogic()</i> .
<i>UFUNCTION(BlueprintCa llable, Category = "Utility Component") const UABUtilityActionBase* GetCurrentAction() const</i>		Returns the current Action.
<i>UFUNCTION(BlueprintCa llable, Category = "Utility Component") const UABUtilityActionBase* GetLastSuccessfulAction() const</i>		Returns the last successful Action.
<i>UFUNCTION(BlueprintCa llable, Category = "Utility Component") const EABActionSelectionMode & GetActionSelectionMode() const</i>		Returns the set Action Selection Mode.
<i>const FTimerHandle& GetSuccessfulActionCool downHandle() const</i>		Returns the Timer Handle for the cooldown set by the last successful Action.
<i>const FAJBUtilityComponentAs yncData& GetAsyncData() const</i>		Returns the read-only asynchronous data for the component.

<i>FAJBUtilityComponentAsyncData& GetMutableAsyncData()</i>		Returns the read-write asynchronous data for the component.
<i>UFUNCTION(BlueprintCallable, Category = "Utility Component") virtual void AbortCurrentAction(const EABUtilityAbortActionLogic& AbortLogic)</i>	<i>AbortLogic</i> - Allow the user to specify what happens to the component when we abort the current Action - stopping the component, pausing the component, or choosing a new Action	Aborts the current Action in different ways depending on the logic option passed in. <i>StopComponent</i> will run <i>StopLogic()</i> . <i>PauseLogicPauseAction</i> will run <i>PauseLogicWithActionDecision()</i> with the option <i>PauseAction</i> . <i>PauseLogicAbortAction</i> will run <i>PauseLogicWithActionDecision()</i> with the option <i>AbortAction</i> . <i>ChooseNewAction</i> will abort the current Action if there is one and then attempt to find and run a new Action.
<i>UFUNCTION(BlueprintCallable, Category = "Utility Component") virtual void PauseLogicWithActionDecision(const FString& Reason,const EABUtilityComponentPauseActionLogic& ActionDecision)</i>	<i>Reason</i> - Allow the user to specify why we are pausing the logic <i>ActionDecision</i> - Allow the user to specify what happens to the current Action upon pausing the component	If there is a valid current Action, runs different logic on this Action depending on the option passed in (<i>PauseAction</i> will run <i>PauseAction()</i> and <i>AbortAction</i> will run <i>AbortAction()</i>), then runs <i>PauseLogic()</i> .
<i>UFUNCTION(BlueprintCallable, Category = "Utility Component") const bool ShouldRestartLogicWhenResumingComponentLogic() const</i>		Returns the value of <i>bRestartLogicWhenResumingComponentLogic</i> .
<i>UFUNCTION(BlueprintCallable, Category = "Utility Component") const bool IsComponentRunningAsy</i>		Returns the value of <i>bRunComponentAsynchr</i> <i>onously</i> .

<i>nchronously() const</i>		
<i>UFUNCTION(BlueprintCallable, Category = "Utility Component") const bool IsUsingParallelRequirementChecking() const</i>		Returns the value of <i>bUseParallelRequirementChecking</i> .
<i>UFUNCTION(BlueprintCallable, Category = "Utility Component") const bool ShouldRetryOnFailure() const</i>		Returns the value of <i>bRetryOnFailure</i> .
<i>UFUNCTION(BlueprintCallable, Category = "Utility Component") const bool IsUsingConsiderationFactor() const</i>		Returns the value of <i>bUseConsiderationFactorScoreModifier</i> .
<i>void GetLifetimeReplicatedProps(TArray<FLifetimeProperty>& OutLifetimeProps) const override</i>	<i>OutLifetimeProps</i> - Used to track a property that is marked to be replicated for the lifetime of the actor channel. This doesn't mean the property will necessarily always be replicated, it just means: "check this property for replication for the life of the actor, and I don't want to think about it anymore". A secondary condition can also be used to skip replication based on the condition results	Sets up the current Action, the last successful Action, and the current run status to be replicated.
<i>UFUNCTION(BlueprintNativeEvent) void OnRep_CurrentAction()</i>		To be implemented in Blueprints to handle the replication logic for the current Action.
<i>UFUNCTION(BlueprintNativeEvent) void OnRep_LastSuccessfulAction()</i>		To be implemented in Blueprints to handle the replication logic for the last successful Action.

<i>UFUNCTION(BlueprintNativeEvent)</i> <i>void</i> <i>OnRep_CurrentRunStatus()</i>		To be implemented in Blueprints to handle the replication logic for the current run status.
<i>virtual void</i> <i>OnRep_CurrentAction_Implementation()</i>		To be overridden and implemented in project specific Utility Components to handle the replication logic for the current Action.
<i>virtual void</i> <i>OnRep_LastSuccessfulAction_Implementation()</i>		To be overridden and implemented in project specific Utility Components to handle the replication logic for the last successful Action.
<i>virtual void</i> <i>OnRep_CurrentRunStatus_Implementation()</i>		To be overridden and implemented in project specific Utility Components to handle the replication logic for the current run status.
<i>virtual void</i> <i>DescribeSelfToGameplayDebugger(FGameplayDebuggerCategory* DebuggerCategory, const int32 ActionIndex) const</i>	<i>DebuggerCategory</i> - The instance of the <i>FGameplayDebuggerCategory_UTILITY</i> currently being used in the Gameplay Debugger Menu <i>ActionIndex</i> - The current index of the available Actions to show the debugging for	Prints the values of the components properties to the gameplay debugger, runs <i>DescribeSelfToGameplayDebugger()</i> on the Action specified by the <i>ActionIndex</i> , and draws a point at the location of the debug actor.
<i>UFUNCTION()</i> <i>virtual void</i> <i>OnActionCompleted(UABUtilityActionBase* Action, const EABUtilityActionRunStatus ActionStatus, const UABUtilityActionInstanceMemoryBase* ActionMemory)</i>	<i>Action</i> - The Action that triggered this instance of the callback <i>ActionStatus</i> - The final status of the Action for this component <i>ActionMemory</i> - The Instance Memory for the	If the <i>ActionStatus</i> is <i>Succeeded</i> , stores the <i>Action</i> as the last successful Action and sets the cooldown timer as the cooldown value grabbed from the <i>Action</i> . Otherwise, logs and tries to find and run a new Action next frame.

	component for the given Action	
<i>void OnActionRequirementsChecked(UABUtilityActionBase* Action, const bool bRequirementsPassed)</i>	<i>Action</i> - The Action that triggered this instance of the callback <i>bRequirementsPassed</i> - Whether or not all of the Actions Requirements passed and therefore if the Action is possible right now	If <i>bRequirementsPassed</i> is true, gets the score for the <i>Action</i> and, if it's not 0, stores the <i>Action</i> and its score in the <i>AsyncData</i> . Once all Actions have been checked, sorts the Actions by score, and then picks one according to the <i>ActionSelectionMode</i> . If all Actions fail, attempts to retry.
<i>bool PerformUtilityAction()</i>		Runs either <i>RequestUtilityAction()</i> or <i>AsyncRequestUtilityAction()</i> depending on the return value of <i>IsComponentRunningAsynchronously()</i> .
<i>bool RequestUtilityAction()</i>		Gets the possible Actions with either <i>GetPossibleActions()</i> or <i>ParallelGetPossibleActions()</i> depending on the return value of <i>IsUsingParallelRequirementChecking()</i> . Sorts the Actions by score, then picks one according to the <i>ActionSelectionMode</i> .
<i>void AsyncRequestUtilityAction()</i>		Runs either <i>AsyncGetPossibleActions()</i> or <i>ParallelAsyncGetPossibleActions()</i> depending on the return value of <i>IsUsingParallelRequirementChecking()</i> .
<i>void GetPossibleActions(TMap<UABUtilityActionBase*,</i>	<i>PossibleActionsToScores</i> - A map with the key as the Action and the value	Gets the available Actions, checks their Requirements, gets their

<i>const float>&PossibleActionsToScores)</i>	as the score for that Action	score, and adds each Action to <i>PossibleActionsToScores</i> if it passes the check and its score is above 0.
<i>void ParallelGetPossibleActions(TMap<UABUtilityActionBase*, const float>&PossibleActionsToScores)</i>	<i>PossibleActionsToScores</i> - A map with the key as the Action and the value as the score for that Action	Gets the available Actions then, within a <i>ParallelFor()</i> helper, checks their Requirements, gets their score, and adds each Action to <i>PossibleActionsToScores</i> if it passes the check and its score is above 0.
<i>void AsyncGetPossibleActions()</i>		Gets the available Actions, and starts an <i>AsyncTask()</i> for each one to run its <i>AsyncCheckAllRequirements()</i> .
<i>void ParallelAsyncGetPossibleActions()</i>		Gets the available Actions then, within a <i>ParallelFor()</i> helper, starts an <i>AsyncTask()</i> for each one to run its <i>AsyncCheckAllRequirements()</i> .
<i>void GetAvailableActionsFromSubsystem(TArray<const UABUtilityActionBase*>&AvailableActions) const</i>	<i>AvailableActions</i> - An array of Actions available to this component instance	Gets the available Actions from the Utility Subsystem.
<i>void GetAvailableMutableActionsFromSubsystem(TArray<UABUtilityActionBase*>&AvailableActions)</i>	<i>AvailableActions</i> - An array of Actions available to this component instance	Gets the available Actions from the Utility Subsystem.
<i>const UABUtilityActionInstanceMemoryBase* GetActionInstanceMemoryFromSubsystem(const UABUtilityActionBase&</i>	<i>Action</i> - The Action to get the Instance Memory for	Gets the Instance Memory from the Utility Subsystem relating to the component and the <i>Action</i> .

<i>Action) const</i>		
<i>UABUtilityActionInstanceMemoryBase*</i> <i>GetMutableActionInstanceMemoryFromSubsystem</i> <i>(const UABUtilityActionBase& Action)</i>	<i>Action</i> - The Action to get the Instance Memory for	Gets the Instance Memory from the Utility Subsystem relating to the component and the <i>Action</i> .
<i>bool AttemptRetry()</i>		If <i>ShouldRetryOnFailure()</i> returns true, sets a timer for next tick to run <i>PerformUtilityAction()</i> .

Members:

Member	Use
<i>UPROPERTY(EditAnywhere, Category = "Utility")</i> <i>TSoftObjectPtr<UABUtilityDataAsset></i> <i>UtilityDataAssetClass</i>	The Utility Data Asset whose Actions the component will subscribe to.
<i>UPROPERTY(ReplicatedUsing = OnRep_CurrentAction)</i> <i>TObjectPtr<UABUtilityActionBase></i> <i>CurrentAction</i>	The Action that the component is currently running.
<i>UPROPERTY(ReplicatedUsing = OnRep_LastSuccessfulAction)</i> <i>TObjectPtr<UABUtilityActionBase></i> <i>LastSuccessfulAction</i>	The last successful Action that the component completed.
<i>UPROPERTY(ReplicatedUsing = OnRep_CurrentRunStatus)</i> <i>EABUtilityActionRunStatus</i> <i>CurrentRunStatus</i>	The current status of the components <i>CurrentAction</i> .
<i>UPROPERTY(EditAnywhere, Category = "Actions")</i> <i>EABActionSelectionMode</i> <i>ActionSelectionMode</i>	The mode of selecting an Action that this component will use.
<i>UPROPERTY(EditAnywhere, Category = "Pausing")</i> <i>uint32</i> <i>bRestartLogicWhenResumingComponentLogic : 1</i>	If true, when <i>ResumeLogic()</i> is run, the component will run <i>RestartLogic()</i> .

<i>UPROPERTY(EditAnywhere, Category = "Async") uint32 bRunComponentAsynchronously : 1</i>	If true, the asynchronous versions of certain functions will run (<i>AsyncRequestUtilityAction()</i>).
<i>UPROPERTY(EditAnywhere, Category = "Async") uint32 bUseParallelRequirementChecking : 1</i>	If true, the parallel versions of certain functions will run (<i>ParallelGetPossibleActions()</i> , <i>ParallelAsyncGetPossibleActions()</i>).
<i>UPROPERTY(EditAnywhere, Category = "Utility") uint32 bRetryOnFailure : 1</i>	If true, when the component fails to run any Action, it will retry.
<i>UPROPERTY(EditAnywhere, Category = "Scoring") uint32 bUseConsiderationFactorScoreModifier</i>	If true, when calculating the score for Actions, the <i>ConsiderationFactor</i> score modifier will be used (https://gdcvault.com/play/1021848/Building-a-Better-Centaur-AI).
<i>FAJBUtilityComponentAsyncData AsyncData</i>	Contains the data required across frames for the asynchronous logic to run correctly.
<i>FTimerHandle SuccessfulActionCooldownHandle</i>	The timer handle for the cooldown after a successful Action has completed.

Other:

Types	Use
<i>struct FAJBUtilityComponentAsyncData</i>	Contains the data required across frames for the asynchronous logic to run correctly.
<i>enum class EABUtilityComponentPauseActionLogic</i>	Defines the options for when the component's logic is paused.

UABUtilityActionBase

This is the base class for all Actions made both in this plugin and by others who may use this plugin. It contains Requirements for checking if it can run at all, Factors for calculating the score for the Action, and Tick functionality for any Actions that require running over multiple frames. Once the Action completes, it will trigger a delegate broadcast that the Utility Component should be tied to that will allow the component to find and run a new Action.

Parent class: *UObject*

Functions:

Function	Params	Use
<i>UABUtilityActionBase()</i>		Constructor.
<i>virtual void Initialize(UWorld* InWorld)</i>	<i>InWorld</i> - The instance of the World to cache and return in <i>GetWorld()</i>	Runs when the Action is loaded and stored in the Utility Subsystem. Binds to the Actions Requirements async check delegate, and caches <i>InWorld</i> .
<i>virtual void Deinitialize()</i>		Runs when the Action is being deleted from the Utility Subsystem. Unbinds from the Actions Requirements async check delegate, and clears <i>CachedWorld</i> .
<i>virtual const TSubclassOf<UABUtilityActionInstanceMemoryBase> GetInstanceMemoryType() const</i>		Returns the class for the Actions Instance Memory. Made to be overridden so it returns the correct class.
<i>UWorld* GetWorld()</i>		Returns <i>CachedWorld</i> .
<i>const bool CheckAllRequirements(co</i>	<i>OwnerComp</i> - The owning Utility Component	Runs either <i>InternalCheckAllRequirem</i>

<i>nst</i> <i>UABUtilityComponent&</i> <i>OwnerComp) const</i>		<i>ents()</i> or <i>InternalParallelCheckAllR</i> <i>equirements()</i> depending on the value returned by <i>IsUsingParallelRequireme</i> <i>ntChecking()</i> .
<i>void</i> <i>AsyncCheckAllRequirements(const</i> <i>UABUtilityComponent&</i> <i>OwnerComp)</i>	<i>OwnerComp</i> - The owning Utility Component	Runs either <i>InternalAsyncCheckAllRe</i> <i>quirements()</i> or <i>InternalParallelAsyncChe</i> <i>ckAllRequirements()</i> depending on the value returned by <i>IsUsingParallelRequireme</i> <i>ntChecking()</i> .
<i>const</i> <i>TArray<UABUtilityRequire</i> <i>mentBase*>&</i> <i>GetRequirements() const</i>		Returns the list of Requirements for this Action.
<i>const</i> <i>TArray<UABUtilityFactorB</i> <i>ase*>& GetFactors()</i> <i>const</i>		Returns the list of Factors for this Action.
<i>const float</i> <i>GetOnSuccessCooldown</i> <i>Value(UABUtilityCompon</i> <i>ent& OwnerComp) const</i>	<i>OwnerComp</i> - The owning Utility Component	Returns the bound value for the <i>OnSuccessCooldown</i> .
<i>const float</i> <i>GetUtilityWeightValue(const</i> <i>UABUtilityComponent&</i> <i>OwnerComp) const</i>	<i>OwnerComp</i> - The owning Utility Component	Returns the bound value for the <i>UtilityWeight</i> .
<i>const float</i> <i>GetActionScore(const</i> <i>UABUtilityComponent&</i> <i>OwnerComp, const bool</i> <i>bUseConsiderationFactor</i> <i>) const</i>	<i>OwnerComp</i> - The owning Utility Component <i>bUseConsiderationFactor</i> - Whether or not the scoring will be modified by the Consideration Factor	Calculates the full score from the Factors, multiplies it by the value returned by <i>GetUtilityWeightValue()</i> , and returns this final value.
<i>const</i> <i>EABUtilityActionRunStatu</i> <i>s</i> <i>PerformAction(UABUtility</i>	<i>OwnerComp</i> - The owning Utility Component	Runs <i>DoAction()</i> . If this does not return <i>Running</i> , completes immediately, otherwise, if the Action

<i>Component& OwnerComp, UABUtilityActionInstance MemoryBase* InstanceMemory)</i>		should tick, runs <i>SetNeedsTick()</i> on the <i>InstanceMemory</i> passing in <i>true</i> .
<i>virtual void OnActionCompleted(UAB UtilityActionInstanceMem oryBase* InstanceMemory, const EABUtilityActionRunStatu s ActionStatus)</i>	<i>InstanceMemory</i> - The Instance Memory for the component that completed the Action <i>ActionStatus</i> - The final status of the Action for a given component	Resets the <i>InstanceMemory</i> if it's valid, and broadcasts the completion <i>ActionStatus</i> .
<i>virtual void PauseAction(UABUtilityC omponent* OwnerComp, const FString& Reason, UABUtilityActionInstance MemoryBase* InstanceMemory = nullptr)</i>	<i>OwnerComp</i> - The owning Utility Component <i>Reason</i> - The reason the Action is being paused <i>InstanceMemory</i> - The Instance Memory for the <i>OwnerComp</i>	Logs and turns the tick off for the <i>InstanceMemory</i> .
<i>virtual bool ResumeAction(UABUtility Component* OwnerComp, UABUtilityActionInstance MemoryBase* InstanceMemory = nullptr)</i>	<i>OwnerComp</i> - The owning Utility Component <i>InstanceMemory</i> - The Instance Memory for the <i>OwnerComp</i>	Logs and, if the Action should tick, turns the tick on for the <i>InstanceMemory</i> .
<i>virtual void AbortAction(UABUtilityCo mponent* OwnerComp, const FString& Reason, UABUtilityActionInstance MemoryBase* InstanceMemory = nullptr)</i>	<i>OwnerComp</i> - The owning Utility Component <i>Reason</i> - The reason the Action is being aborted <i>InstanceMemory</i> - The Instance Memory for the <i>OwnerComp</i>	Logs and runs <i>OnActionCompleted()</i> .
<i>virtual EABUtilityActionRunStatu s TickAction(float DeltaTime, UABUtilityComponent& OwnerComp, UABUtilityActionInstance</i>	<i>DeltaTime</i> - The Tick time for this frame <i>OwnerComp</i> - The component that needs this Action to tick	Runs <i>Tick()</i> and, if the Action no longer needs to tick, runs <i>OnActionCompleted()</i> .

<i>MemoryBase*</i> <i>InstanceMemory)</i>	<i>InstanceMemory</i> - The Instance Memory for the <i>OwnerComp</i>	
<i>const bool</i> <i>IsUsingParallelRequirementChecking() const</i>		Returns the value of <i>bUseParallelRequirementChecking</i> .
<i>const bool</i> <i>IsConsideringFactorScoreZero() const</i>		Returns the value of <i>bConsiderFactorScoreZero</i> .
<i>const bool ShouldTick() const</i>		Returns the value of <i>bShouldTick</i> .
<i>virtual const bool</i> <i>ValidateAction(FString& InvalidationReason) const</i>	<i>InvalidationReason</i> - The string to be printed if anything about the Action is invalid	Checks that <i>UtilityWeight</i> and <i>OnSuccessCooldown</i> are both above or equal to 0.
<i>virtual void</i> <i>DescribeSelfToGameplayDebugger(const UABUtilityComponent& OwnerComp, const bool bUseConsiderationFactor, const UABUtilityActionInstance MemoryBase* InstanceMemory, FGameplayDebuggerCategory* DebuggerCategory) const</i>	<i>OwnerComp</i> - The owning Utility Component <i>bUseConsiderationFactor</i> - Whether or not the score will be modified by the consideration factor <i>InstanceMemory</i> - The Instance Memory for the <i>OwnerComp</i> <i>DebuggerCategory</i> - The instance of the <i>FGameplayDebuggerCategory_UTILITY</i> currently being used in the Gameplay Debugger Menu	Prints the value of the Actions properties to the gameplay debugger, and runs <i>DescribeSelfToGameplayDebugger()</i> on all the Actions Requirements and Factors.
<i>virtual</i> <i>EABUtilityActionRunStatus</i> <i>DoAction(UABUtilityComponent& OwnerComp, UABUtilityActionInstance MemoryBase* InstanceMemory)</i>	<i>OwnerComp</i> - The owning Utility Component <i>InstanceMemory</i> - The Instance Memory for the <i>OwnerComp</i>	To be overridden by Actions inheriting from <i>UABUtilityActionBase</i> . Action logic should live here.
<i>virtual</i>	<i>OwnerComp</i> - The	To be overridden by

<i>EABUtilityActionRunStatus</i> <i>Tick(UABUtilityComponent& OwnerComp, float DeltaSeconds, UABUtilityActionInstanceMemoryBase* InstanceMemory)</i>	owning Utility Component <i>DeltaSeconds</i> - The Tick time for this frame <i>InstanceMemory</i> - The Instance Memory for the <i>OwnerComp</i>	Actions inheriting from <i>UABUtilityActionBase</i> . Action tick logic should live here.
<i>const bool InternalCheckAllRequirements(const UABUtilityComponent& OwnerComp) const</i>	<i>OwnerComp</i> - The owning Utility Component	Iterates through each of the Actions Requirements and checks them. Stops early if one of them fails.
<i>const bool InternalParallelCheckAllRequirements(const UABUtilityComponent& OwnerComp) const</i>	<i>OwnerComp</i> - The owning Utility Component	Within a <i>ParallelFor()</i> helper, checks each requirement. Returns false if any of the Requirements fail.
<i>void InternalAsyncCheckAllRequirements(const UABUtilityComponent& OwnerComp)</i>	<i>OwnerComp</i> - The owning Utility Component	Iterates through each of the Actions Requirements and sets up an <i>AsyncTask()</i> for them that runs <i>AsyncCheckRequirement()</i> .
<i>void InternalParallelAsyncCheckAllRequirements(const UABUtilityComponent& OwnerComp)</i>	<i>OwnerComp</i> - The owning Utility Component	Within a <i>ParallelFor()</i> helper, sets up an <i>AsyncTask()</i> for each Requirement that runs <i>AsyncCheckRequirement()</i> .
<i>virtual void OnAsyncRequirementChecked(const UABUtilityRequirementBase* const Requirement, const bool bRequirementPassed)</i>	<i>Requirement</i> - The Requirement that triggered this instance of the callback <i>bRequirementPassed</i> - Whether or not the Requirement passed	Updates the <i>AsyncData</i> to contain the value of all Requirements "AND"d together. Once all Requirements have been checked, broadcasts the final value on the game thread.
<i>void ModifyTotalScore(const float& ModifyingScore, const float& NumFactors, const int32&</i>	<i>ModifyingScore</i> - The score to modify the total by <i>NumFactors</i> - The	Modifies the Actions total score in that frame by the score of the Factor just checked. The method used to modify the score

<i>CurrentIndex, float& OutTotalScore) const</i>	number of Factors held by the Action as a <i>float</i> <i>CurrentIndex</i> - The index of the current Factor being calculated <i>OutTotalScore</i> - The score after the modifier has been applied	is set by <i>ScoreAggregationMethod</i> .
<i>virtual const bool ShouldTickIfViewportsOnly() const</i>		Returns <i>bShouldTickIfViewportsOnly</i> .

Members:

Member	Use
<i>FOnActionCompletedDelegate OnActionCompletedDelegate</i>	Delegate broadcast when the Action completes. Data broadcast is the Action, the Action status, and the Instance Memory.
<i>FOnAsyncRequirementsChecked OnAsyncRequirementsCheckedDelegate</i>	Delegate broadcast when all async Requirements have been checked. Data broadcast is the Action and the final result of all Requirements together.
<i>TObjectPtr<UWorld> CachedWorld</i>	The instance of the world held by an Action returned in <i>GetWorld()</i> .
<i>FAJBUtilityActionAsyncData AsyncData</i>	Contains the data required across frames for the asynchronous logic to run correctly.
<i>UPROPERTY(EditAnywhere, meta = (ClampMin = 0.0, UIMin = 0.0)) FAIDataProviderFloatValue UtilityWeight</i>	The weight multiplier that gets multiplied in at the end of scoring the Action.
<i>UPROPERTY(EditAnywhere, meta = (ClampMin = 0.0, UIMin = 0.0)) FAIDataProviderFloatValue OnSuccessCooldown</i>	The cooldown time the component will wait for after this Action was successful.
<i>UPROPERTY(EditAnywhere) EABFactorScoreTotallingMethod ScoreAggregationMethod</i>	The method used to combine all Factor scores.

<i>UPROPERTY(EditAnywhere, Instanced)</i> <i>TArray<TObjectPtr<UABUtilityRequirementBase>> Requirements</i>	The list of Requirements that need to be passed for this Action to be able to run.
<i>UPROPERTY(EditAnywhere, Instanced)</i> <i>TArray<TObjectPtr<UABUtilityFactorBase>> Factors</i>	The list of Factors that will affect this Actions score.
<i>UPROPERTY(EditAnywhere)</i> <i>uint32</i> <i>bUseParallelRequirementChecking : 1</i>	If true, certain parallel functions will be used (<i>InternalParallelCheckAllRequirements()</i> and <i>InternalParallelAsyncCheckAllRequirements()</i>).
<i>UPROPERTY(EditAnywhere)</i> <i>uint32 bConsiderFactorScoreZero : 1</i>	If true, if a Factor score is 0 it will still be considered.
<i>UPROPERTY(EditAnywhere, Category = "Utility AI Action Base")</i> <i>uint32 bShouldTick : 1</i>	If true, the Action will tick for any component that is running it and needs it to tick.
<i>UPROPERTY(EditAnywhere)</i> <i>bool bShouldTickIfViewportsOnly</i>	If true, the Action will still tick even if the <i>TickType</i> is <i>LEVELTICK_ViewportsOnly</i> .

Other:

Types	Use
<i>struct FAJBUtilityActionAsyncData</i>	Contains the data required across frames for the asynchronous logic to run correctly.
<i>enum class</i> <i>EABFactorScoreTotallingMethod</i>	Defines the method used to combine all the Factor scores together when calculating the Action score.

UABUtilityRequirementBase

Requirements are used to check if an Action is able to run at that moment. When the Utility Component tries to run a new Action, it checks the Requirements of that Action first. They must all pass for the Action to be possible when checked. The only case where not every Requirement needs to pass is if the user adds a Multi-Requirement to the Action that groups with an OR operator instead of an AND operator.

Parent class: *UObject*

Functions:

Function	Params	Use
<i>virtual const bool CheckRequirement(const UABUtilityComponent& OwnerComp) const</i>	<i>OwnerComp</i> - The owning Utility Component	To be overridden by Requirements inheriting from <i>UABUtilityRequirementBase</i> . The logic should live here.
<i>void AsyncCheckRequirement(const UABUtilityComponent& OwnerComp) const</i>	<i>OwnerComp</i> - The owning Utility Component	Runs <i>CheckRequirement()</i> then broadcasts the result on the game thread.
<i>UWorld* GetWorld() const override</i>		Returns the result of <i>GetOuter()->GetWorld()</i> .
<i>virtual const bool ValidateRequirement(FString& InvalidationReason) const</i>	<i>InvalidationReason</i> - The string to be printed if anything about the Requirement is invalid	To be overridden by Requirements inheriting from <i>UABUtilityRequirementBase</i> that have properties that should be validated at editor time.
<i>virtual void DescribeSelfToGameplayDebugger(const UABUtilityComponent& OwnerComp,</i>	<i>OwnerComp</i> - The owning Utility Component <i>DebuggerCategory</i> - The instance of the	Prints the current result of the Requirement to the gameplay debugger.

<i>FGameplayDebuggerCategory*</i> <i>DebuggerCategory) const</i>	<i>FGameplayDebuggerCategory_Utility</i> currently being used in the Gameplay Debugger Menu	
---	---	--

Members:

Member	Use
<i>FOnAsyncRequirementChecked</i> <i>OnAsyncRequirementCheckedDelegate</i>	Delegate broadcast when the async Requirement has been checked. Data broadcast is the Requirement and the result.

UABUtilityFactorBase

Factors are used to calculate the score for an Action when the Utility Component has already checked its Requirements and is considering it as a possibility. Each Factor has its own score that is evaluated when the Action is having its score calculated, then all of an Actions scores are totalled using a user-set method. If a Factors score returns 0, it will usually be ignored unless the user has enabled

bConsiderFactorScoreZero on the owning Action. The Factor has an optional extra bit of functionality called the “Consideration Factor”, which is inspired by and

described well in this video: [GDC Vault - Building a Better Centaur: AI at Massive Scale](#)

Parent class: *UObject*

Functions:

Function	Params	Use
<i>const float</i> <i>GetFactorScore(const UABUtilityComponent& OwnerComp, const float NumberOfFactors, const bool bUseConsiderationFactor) const</i>	<i>OwnerComp</i> - The owning Utility Component <i>NumberOfFactors</i> - The number of Factors owned by the owning Action, used by the Consideration Factor <i>bUseConsiderationFactor</i> - Whether or not the score will be modified by the Consideration Factor	Runs <i>CalculateFactorScore()</i> then, if <i>bUseConsiderationFactor</i> is true, modifies the score using the Consideration Factor. Returns the final result.
<i>const bool IsInversed()</i> <i>const</i>		Returns the value of <i>bInverseScore</i> .
<i>UWorld* GetWorld() const override</i>		Returns the value of <i>GetOuter()->GetWorld()</i> .
<i>virtual const bool ValidateFactor(FString& InvalidationReasons)</i>	<i>InvalidationReason</i> - The string to be printed if anything about the Factor	To be overridden by Factors inheriting from <i>UABUtilityFactorBase</i> that

<i>const</i>	is invalid	have properties that should be validated at editor time.
<i>virtual void DescribeSelfToGameplay Debugger(const UABUtilityComponent& OwnerComp, const float NumberOfFactors, const bool bUseConsiderationFactor, FGameplayDebuggerCategory* DebuggerCategory) const</i>	<i>OwnerComp</i> - The owning Utility Component <i>NumberOfFactors</i> - The number of Factors owned by the owning Action, is used by the Consideration Factor <i>bUseConsiderationFactor</i> - Whether or not the score will be modified by the Consideration Factor <i>DebuggerCategory</i> - The instance of the <i>FGameplayDebuggerCategory_UTILITY</i> currently being used in the Gameplay Debugger	Prints the value of <i>bInverseScore</i> , <i>bUseConsiderationFactor</i> and the Factor score to the gameplay debugger.
<i>virtual const float CalculateFactorScore(const UABUtilityComponent& OwnerComp) const</i>	<i>OwnerComp</i> - The owning Utility Component	To be overridden by Factors inheriting from <i>UABUtilityFactorBase</i> . Factor logic should live here.
<i>void AddConsiderationFactor(const float NumberOfFactors, float& OutFloat) const</i>	<i>NumberOfFactors</i> - The number of Factors owned by the owning Action <i>OutFloat</i> - The Factor score to be used in the calculation and set at the end of the logic	Modifies the Factor Score by the Consideration Factor.

Members:

Member	Use
<i>UPROPERTY(EditAnywhere, Category = "Scoring") uint32 bInverseScore : 1</i>	Reverses the Factor score by subtracting it from 1 (Factor scores should be normalised).

UABUtilityActionBlueprintBase

This Action is the base for any Actions that users wish to make in Blueprints instead of C++. It overrides the relevant functions from the Action base, and adds Blueprint functions that allow users to validate, debug, run and Tick an Action.

Parent class: *UABUtilityActionBase*

Functions:

Function	Params	Use
<i>const bool ValidateAction(FString& InvalidationReason) const override</i>	<i>InvalidationReason</i> - The string to be printed if anything about the Action is invalid	Calls and returns the combined value of <i>Super::ValidateAction()</i> and <i>ValidateBlueprintAction()</i> .
<i>UFUNCTION(BlueprintNativeEvent) const bool ValidateBlueprintAction(const FString& InInvalidationReason, FString& OutInvalidationReason) const</i>	<i>InInvalidationReason</i> - The string that already exists to be printed <i>OutInvalidationReason</i> - The string to be printed if anything about the Action is invalid, should contain the string passed into <i>InInvalidationReason</i>	To be overridden in Blueprints and used to validate any Blueprint properties.
<i>virtual const bool ValidateBlueprintActionImplementation(const FString& InInvalidationReason, FString& OutInvalidationReason) const</i>	<i>InInvalidationReason</i> - The string that already exists to be printed <i>OutInvalidationReason</i> - The string to be printed if anything about the Action is invalid, should contain the string passed into <i>InInvalidationReason</i>	To be overridden by Actions inheriting from <i>UABUtilityActionBlueprintBase</i> and used to validate Action properties.
<i>void DescribeSelfToGameplayDebugger(const UABUtilityComponent& OwnerComp, const bool bUseConsiderationFactor, const</i>	<i>OwnerComp</i> - The owning Utility Component <i>bUseConsiderationFactor</i> - Whether or not the score will be modified by the Consideration Factor	Prints the value returned by <i>DescribeBlueprintSelfToGameplayDebugger()</i> to the gameplay debugger.

<i>UABUtilityActionInstance MemoryBase*</i> <i>InstanceMemory,</i> <i>FGameplayDebuggerCat egory*</i> <i>DebuggerCategory)</i> const override	<i>InstanceMemory</i> - The Instance Memory for the OwnerComp <i>DebuggerCategory</i> - The instance of the <i>FGameplayDebuggerCat egory_Utility</i> currently being used in the Gameplay Debugger Menu	
<i>UFUNCTION(BlueprintNa tiveEvent)</i> const FString DescribeBlueprintSelfToG ameplayDebugger(const UABUtilityComponent* const OwnerComp) const	OwnerComp - The owning Utility Component	To be overridden in Blueprints to print any Blueprint property values to the gameplay debugger.
virtual const FString DescribeBlueprintSelfToG ameplayDebugger_Imple mentation(const UABUtilityComponent* const OwnerComp) const	OwnerComp - The owning Utility Component	To be overridden by Actions inheriting from <i>UABUtilityActionBlueprint Base</i> and used to print Action properties.
<i>EABUtilityActionRunStatu s</i> DoAction(UABUtilityComp onent& OwnerComp, UABUtilityActionInstance MemoryBase* InstanceMemory) override	OwnerComp - The owning Utility Component <i>InstanceMemory</i> - The Instance Memory for the OwnerComp	Runs <i>DoBlueprintAction()</i> .
<i>UFUNCTION(BlueprintNa tiveEvent)</i> <i>EABUtilityActionRunStatu s</i> DoBlueprintAction(UABUt ilityComponent* OwnerComp)	OwnerComp - The owning Utility Component	To be overridden in Blueprints. Blueprint Action logic should live here.
virtual <i>EABUtilityActionRunStatu s</i> DoBlueprintAction_Imple mentation(UABUtilityCom ponent* OwnerComp)	OwnerComp - The owning Utility Component	To be overridden in Actions inheriting from <i>UABUtilityActionBlueprint Base</i> and used to define Action logic.

<i>EABUtilityActionRunStatus</i> <i>Tick(UABUtilityComponent* OwnerComp, float DeltaSeconds, UABUtilityActionInstanceMemoryBase* InstanceMemory) override</i>	<i>OwnerComp</i> - The owning Utility Component <i>DeltaSeconds</i> - The Tick time in seconds for this frame <i>InstanceMemory</i> - The Instance Memory for the <i>OwnerComp</i>	Runs <i>TickBlueprintAction()</i> .
<i>UFUNCTION(BlueprintNativeEvent)</i> <i>EABUtilityActionRunStatus</i> <i>TickBlueprintAction(UABUtilityComponent* OwnerComp, float DeltaSeconds)</i>	<i>OwnerComp</i> - The owning Utility Component <i>DeltaSeconds</i> - The Tick time in seconds for this frame	To be overridden in Blueprints and used to define Action tick logic.
<i>virtual</i> <i>EABUtilityActionRunStatus</i> <i>TickBlueprintAction_Implementation(UABUtilityComponent* OwnerComp, float DeltaSeconds)</i>	<i>OwnerComp</i> - The owning Utility Component <i>DeltaSeconds</i> - The Tick time in seconds for this frame	To be overridden in Actions inheriting from <i>UABUtilityActionBlueprintBase</i> and used to define Action tick logic.

UABUtilityRequirementBlueprintBase

This Requirement is the base for any Requirements that users wish to make in Blueprint instead of C++. It overrides the relevant functions from the Requirement base, and adds Blueprint functions that allow users to validate, debug and check the Requirement.

Parent class: *UABUtilityRequirementBase*

Functions:

Function	Params	Use
<i>const bool CheckRequirement(const UABUtilityComponent& OwnerComp) const override</i>	<i>OwnerComp</i> - The owning Utility Component	Runs <i>CheckBlueprintRequirement()</i> .
<i>const bool ValidateRequirement(FString& InvalidationReason) const override</i>	<i>InvalidationReason</i> - The string to be printed if anything about the Requirement is invalid	Calls and returns the combined value of <i>Super::ValidateRequirement()</i> and <i>ValidateBlueprintRequirement()</i> .
<i>const bool ValidateBlueprintRequirement(const FString& InInvalidationReason, FString& OutInvalidationReason) const</i>	<i>InInvalidationReason</i> - The string that already exists to be printed <i>OutInvalidationReason</i> - The string to be printed if anything about the Requirement is invalid, should contain the string passed into <i>InInvalidationReason</i>	To be overridden in Blueprints and used to validate any Blueprint properties at editor time.
<i>virtual const bool ValidateBlueprintRequirement_Implementation(const FString& InInvalidationReason, FString& OutInvalidationReason) const</i>	<i>InInvalidationReason</i> - The string that already exists to be printed <i>OutInvalidationReason</i> - The string to be printed if anything about the Requirement is invalid,	To be overridden in Requirements inheriting from <i>UABUtilityRequirementBlueprintBase</i> and used to validate any Requirement properties at editor time.

	should contain the string passed into <i>InInvalidationReason</i>	
<i>void DescribeSelfToGameplayDebugger(const UABUtilityComponent& OwnerComp, FGameplayDebuggerCategory* DebuggerCategory) const override</i>	<i>OwnerComp</i> - The owning Utility Component <i>DebuggerCategory</i> - The instance of the <i>FGameplayDebuggerCategory_UTILITY</i> currently being used in the Gameplay Debugger Menu	Prints the value returned by <i>DescribeBlueprintSelfToGameplayDebugger()</i> to the gameplay debugger.
<i>UFUNCTION(BlueprintNativeEvent) const FString DescribeBlueprintSelfToGameplayDebugger(const UABUtilityComponent* const OwnerComp) const</i>	<i>OwnerComp</i> - The owning Utility Component	To be overridden in Blueprints and used to print any Blueprint property values to the gameplay debugger.
<i>virtual const FString DescribeBlueprintSelfToGameplayDebugger_Implementation(const UABUtilityComponent* const OwnerComp) const</i>	<i>OwnerComp</i> - The owning Utility Component	To be overridden in any Requirements inheriting from <i>UABUtilityRequirementBlueprintBase</i> and used to print any Requirement property values to the gameplay debugger.
<i>UFUNCTION(BlueprintNativeEvent) const bool CheckBlueprintRequirement(const UABUtilityComponent* const OwnerComp) const</i>	<i>OwnerComp</i> - The owning Utility Component	To be overridden in Blueprints and used to define the Requirement logic.
<i>virtual const bool CheckBlueprintRequirement_Implementation(const UABUtilityComponent* const OwnerComp) const</i>	<i>OwnerComp</i> - The owning Utility Component	To be overridden in any Requirements inheriting from <i>UABUtilityRequirementBlueprintBase</i> and used to define Requirement logic.

UABUtilityFactorBlueprintBase

This Factor is the base for any Factors that users wish to make in Blueprint instead of C++. It overrides the relevant functions from the Factor base, and adds Blueprint functions that allow users to validate, debug and calculate the Factor score.

Parent class: *UABUtilityFactorBase*

Functions:

Function	Params	Use
<i>const bool ValidateFactor(FString& InvalidationReasons) const override</i>	<i>InvalidationReason</i> - The string to be printed if anything about the Factor is invalid	Calls and returns the combined value of <i>Super::ValidateFactor()</i> and <i>ValidateBlueprintFactor()</i> .
<i>UFUNCTION(BlueprintNativeEvent) const bool ValidateBlueprintFactor(const FString& InInvalidationReason, FString& OutInvalidationReason) const</i>	<i>InInvalidationReason</i> - The string that already exists to be printed <i>OutInvalidationReason</i> - The string to be printed if anything about the Factor is invalid, should contain the string passed into <i>InInvalidationReason</i>	To be overridden in Blueprints and used to validate any Blueprint properties at editor time.
<i>virtual const bool ValidateBlueprintFactorImplementation(const FString& InInvalidationReason, FString& OutInvalidationReason) const</i>	<i>InInvalidationReason</i> - The string that already exists to be printed <i>OutInvalidationReason</i> - The string to be printed if anything about the Factor is invalid, should contain the string passed into <i>InInvalidationReason</i>	To be overridden in any Factors inheriting from <i>UABUtilityFactorBlueprintBase</i> and used to validate any Factor properties at editor time.
<i>void DescribeSelfToGameplayDebugger(const UABUtilityComponent& OwnerComp, const float NumberOfFactors, const bool</i>	<i>OwnerComp</i> - The owning Utility Component <i>NumberOfFactors</i> - The number of Factors owned by the owning Action	Prints the value returned by <i>DescribeBlueprintSelfToGameplayDebugger()</i> to the gameplay debugger.

<i>bUseConsiderationFactor</i> , <i>FGameplayDebuggerCategory*</i> <i>DebuggerCategory</i>) <i>const override</i>	<i>bUseConsiderationFactor</i> - Whether or not the score will be modified by the Consideration Factor <i>DebuggerCategory</i> - The instance of the <i>FGameplayDebuggerCategory_UTILITY</i> currently being used in the Gameplay Debugger Menu	
<i>UFUNCTION(BlueprintNativeEvent)</i> <i>const FString DescribeBlueprintSelfToGameplayDebugger(const UABUtilityComponent* const OwnerComp, const float NumberOfFactors)</i> <i>const</i>	<i>OwnerComp</i> - The owning Utility Component <i>NumberOfFactors</i> - The number of Factors owned by the owning Action	To be overridden in Blueprints and used to print any Blueprint property values to the gameplay debugger.
<i>virtual const FString DescribeBlueprintSelfToGameplayDebugger_Implementation(const UABUtilityComponent* const OwnerComp, const float NumberOfFactors)</i> <i>const</i>	<i>OwnerComp</i> - The owning Utility Component <i>NumberOfFactors</i> - The number of Factors owned by the owning Action	To be overridden in any Factors inheriting from <i>UABUtilityFactorBlueprintBase</i> and used to print any Factor property values to the gameplay debugger.
<i>const float CalculateFactorScore(const UABUtilityComponent& OwnerComp)</i> <i>const override</i>	<i>OwnerComp</i> - The owning Utility Component	Runs <i>CalculateBlueprintFactorScore()</i> .
<i>UFUNCTION(BlueprintNativeEvent)</i> <i>const float CalculateBlueprintFactorScore(const UABUtilityComponent* const OwnerComp)</i> <i>const</i>	<i>OwnerComp</i> - The owning Utility Component	To be overridden in Blueprints and used to define the Factor logic.
<i>virtual const float CalculateBlueprintFactorScore_Implementation(const UABUtilityComponent*</i>	<i>OwnerComp</i> - The owning Utility Component	To be overridden in any Factors inheriting from <i>UABUtilityFactorBlueprintBase</i> and used to define

<i>const OwnerComp) const</i>		Factor logic.
-------------------------------	--	---------------

UABUtilityDataAsset

This is the data asset used to define the Actions available to an AI. It allows users to define a list of Actions and fill out the data for those Actions, as well as define lists of Requirements and Factors for those Actions and fill out the data for those as well.

Parent class: *UDataAsset*

Functions:

Function	Params	Use
<i>const</i> <i>TArray<TObjectPtr<UABUtilityActionBase>>&</i> <i>GetActions() const</i>		Returns the list of Actions defined in this asset.

Members:

Member	Use
<i>UPROPERTY(EditAnywhere, Instanced)</i> <i>TArray<TObjectPtr<UABUtilityActionBase>> UtilityActions</i>	The list of Actions available to any component referencing this asset.

UABUtilitySettings

This is a Developer Settings class for the Utility plugin, containing assets that cannot be exposed on the subsystem and other global variables.

Parent class: *UDeveloperSettings*

Functions:

Function	Params	Use
<code>const TSet<const TSoftObjectPtr<const UABUtilityDataAsset>>& GetUtilityDataAssets() const</code>		Returns the list of assets to have their Actions loaded at runtime.
<code>const bool ShouldPrintCallstackInLogs() const</code>		Returns the value of <i>bPrintCallstackInLogs</i> .

Members:

Member	Use
<code>UPROPERTY(EditAnywhere, Config, Category = "Data Assets") TSet<const TSoftObjectPtr<const UABUtilityDataAsset>> UtilityDataAssets</code>	The list of assets to have their Actions loaded and made available at runtime.
<code>UPROPERTY(EditAnywhere, Config, Category = "Debugging") bool bPrintCallstackInLogs = true</code>	If true, the callstack will be grabbed and logged when using <i>ABUtility::Helpers::UtilityVLOG()</i> . Grabbing the callstack is a slow operation, only have this enabled for debugging and developer purposes, not for production.

UABUtilitySubsystem

This subsystem holds all instances of Actions and Instance Memory, and allows Utility Components to subscribe to Actions by supplying an unloaded Data Asset that the subsystem gets the Actions from, creates Instance Memory for, and stores locally.

Parent class: *UTickableWorldSubsystem*

Functions:

Function	Params	Use
<i>void Initialize(FSubsystemCollectionBase& Collection) override</i>	<i>Collection</i> - The collection of subsystems	Overridden from <i>UTickableWorldSubsystem</i> and used to load the Actions from the data assets set in the Utility Settings.
<i>void Deinitialize() override</i>		Overridden from <i>UTickableWorldSubsystem</i> to unload the cached Actions.
<i>void Tick(float DeltaTime) override</i>	<i>DeltaTime</i> - The Tick time for this frame	Ticks the Actions that need ticking.
<i>TStatId GetStatId() const override</i>		Returns the <i>TStatId</i> for the class <i>UABUtilitySubsystem</i> .
<i>const bool SubscribeToDataAsset(UABUtilityComponent& SubscribingComponent, const TSoftObjectPtr<UABUtilityDataAsset>& DataAssetClass)</i>	<i>SubscribingComponent</i> - The Utility Component subscribing to Actions <i>DataAssetClass</i> - The unloaded Data Asset containing the Actions to subscribe to	Loads the Actions from the <i>DataAssetClass</i> passed in, and creates an Instance Memory instance of the class grabbed from the Actions <i>GetInstanceMemoryType()</i> . Caches the IM and <i>SubscribingComponent</i> together.
<i>const bool Unsubscribe(const UABUtilityComponent&</i>	<i>UnsubscribingComponent</i> - The Utility Component unsubscribing from	Removes the Instance Memory from the cached IM list for each Action that

<i>UnsubscribingComponent</i>)	Actions	has the <i>UnsubscribingComponent</i> subscribed to it.
<i>const TArray<const UABUtilityActionBase*> GetActionsAvailableToComponent(const UABUtilityComponent& Component) const</i>	<i>Component</i> - The Utility Component to get the available Actions for	Returns the list of Actions that have the <i>Component</i> cached with Instance Memory.
<i>TArray<UABUtilityActionBase*> GetMutableActionsAvailableToComponent(const UABUtilityComponent& Component) const</i>	<i>Component</i> - The Utility Component to get the available Actions for	Returns the list of Actions that have the <i>Component</i> cached with Instance Memory.
<i>const UABUtilityActionInstanceMemoryBase* GetActionInstanceMemoryForComponent(const UABUtilityActionBase& Action, const UABUtilityComponent& Component) const</i>	<i>Action</i> - The Action to find the Instance Memory Type for <i>Component</i> - The Component to find Instance Memory for	Runs <i>InternalGetMutableActionInstanceMemoryForComponent()</i> .
<i>UABUtilityActionInstanceMemoryBase* GetMutableActionInstanceMemoryForComponent(const UABUtilityActionBase& Action, const UABUtilityComponent& Component)</i>	<i>Action</i> - The Action to find the Instance Memory Type for <i>Component</i> - The Component to find the Instance Memory for	Runs <i>InternalGetMutableActionInstanceMemoryForComponent()</i> .
<i>const UABUtilityComponent* GetComponentForActionInstanceMemory(const UABUtilityActionBase& Action, const UABUtilityActionInstanceMemoryBase& InstanceMemory) const</i>	<i>Action</i> - The Action to find a subscribed Component for <i>InstanceMemory</i> - The Instance Memory to use to find a specific subscribed Component	Runs <i>GetMutableComponentForActionInstanceMemory()</i> .
<i>UABUtilityComponent* GetMutableComponentFo</i>	<i>Action</i> - The Action to find a subscribed Component	Gets all Instance Memory for the <i>Action</i> , iterates

<i>rActionInstanceMemory(c const UABUtilityActionBase& Action, const UABUtilityActionInstance MemoryBase& InstanceMemory) const</i>	for <i>InstanceMemory</i> - The Instance Memory to use to find a specific subscribed Component	through and returns the component where the Instance Memory matches the <i>InstanceMemory</i> passed in.
<i>bool DoesSupportWorldType(c const EWorldType::Type WorldType) const override</i>	<i>WorldType</i> - The type of world being initialized	Overridden to ensure the subsystem is only initialised if the <i>WorldType</i> is <i>Game</i> or <i>PIE</i> .
<i>UABUtilityActionInstance MemoryBase* InternalGetMutableAction/ InstanceMemoryForComp onent(const UABUtilityActionBase& Action, const UABUtilityComponent& Component) const</i>	<i>Action</i> - The Action to find the Instance Memory Type for <i>Component</i> - The Component to find the Instance Memory for	Gets all Instance Memory for the <i>Action</i> , and returns the IM for the given <i>Component</i> .

Members:

Member	Use
<i>TMap<UABUtilityActionBase*, FInstanceMemoryWrapper> ActionComponentInstanceMemory</i>	The cached Actions and their lists of Instance Memory.

Other:

Types	Use
<i>struct FInstanceMemoryWrapper</i>	Holds a map of Utility Component to Instance Memory, used as a workaround for the fact a map cannot have another map as its value.

FGameplayDebuggerCategory_Utility

The custom Gameplay Debugger Category used for debugging the Utility

Component, Actions, Requirements and Factors.

Parent class: *FGameplayDebuggerCategory*

Functions:

Function	Params	Use
<i>FGameplayDebuggerCategory_Utility()</i>		Constructor. Sets <i>ActionIndex</i> to 0 and binds the key press <i>Shift+I</i> to <i>OnChangeActionIndex()</i> .
<i>void CollectData(APlayerController* OwnerPC, AActor* DebugActor) override</i>	<i>OwnerPC</i> - The player controller for the user that enabled the Gameplay Debugger Menu <i>DebugActor</i> - The actor in the world that is being debugged and having its information displayed in the Gameplay Debugger Menu	Gets the controller from the pawn-casted <i>DebugActor</i> , gets the Utility Component from the controller, and calls <i>DescribeSelfToGameplayDebugger()</i> on the component, passing in <i>ActionIndex</i> .
<i>static TSharedRef<FGameplayDebuggerCategory> MakeInstance()</i>		Creates a single shared instance of <i>FGameplayDebuggerCategory_Utility</i> .
<i>void OnChangeActionIndex()</i>		Increments <i>ActionIndex</i> .

Members:

Member	Use
<i>int32 ActionIndex</i>	Given to the debug component to grab the Action at a specific index.

UABUtilityDataAssetValidator

The editor validation class that validates the Utility Data Asset that has just been saved.

Parent class: *UEditorValidatorBase*

Functions:

Function	Params	Use
<i>bool CanValidateAsset_Implementation(const FAssetData& InAssetData, UObject* InObject, FDataValidationContext& InContext) const override</i>	<i>InAssetData</i> - Holds important information about the asset found by the Asset Registry <i>InObject</i> - The asset that is being validated <i>Context</i> - The interface between <i>UObject::IsDataValid</i> and the data validation system	Overridden from <i>UEditorValidatorBase</i> to ensure the object being validated is of type <i>UABUtilityDataAsset</i> .
<i>EDataValidationResult ValidateLoadedAsset_Implementation(const FAssetData& InAssetData, UObject* InAsset, FDataValidationContext& Context) override</i>	<i>InAssetData</i> - Holds important information about the asset found by the Asset Registry <i>InAsset</i> - The asset that is being validated <i>Context</i> - The interface between <i>UObject::IsDataValid</i> and the data validation system	Gets the Actions from the data asset object, and calls their <i>ValidateAction()</i> , building up strings to be printed in the editor output if any Actions have invalid properties. Does the same for Requirements and Factors, and then combines all the Invalidation Reasons into a single <i>ErrorMessage</i> string.

ABUtilityTypes.h

This is a helper file containing types that exist across multiple classes.

Type	Use
<i>UENUM(BlueprintType)</i> <i>enum class EABUtilityActionRunStatus</i>	Defines the current status of a Utility Component running Actions.
<i>enum class EABUtilityAbortActionLogic</i>	Defines the options for when an Action should be aborted.
<i>UENUM(BlueprintType)</i> <i>enum class EABActionSelectionMode</i>	Defines the methods a component can use for picking an Action.
<i>class</i> <i>UABUtilityActionInstanceMemoryBase</i>	The base class for Instance Memory stored in the Utility Subsystem. Has functionality for ticking and a base <i>Reset()</i> function to be overridden in classes inheriting from <i>UABUtilityActionInstanceMemoryBase</i> and used to reset any memory being stored.

ABUtilityHelpers.h

This is a helper file containing namespaces and classes that are used by multiple classes and/or structs.

Namespaces:

<i>ABUtility::Helpers</i>		
Function	Params	Use
<i>bool CompareValues(const float Left, const float Right, const EGenericAICheck& ComparisonType)</i>	<i>Left</i> - The first value to be compared <i>Right</i> - The second value to be compared <i>ComparisonType</i> - How the values should be compared	Used to compare two values using a <i>ComparisonType</i> .
<i>const UABUtilitySubsystem* GetUtilitySubsystem(const UObject* WorldContextObject)</i>	<i>WorldContextObject</i> - An object with <i>GetWorld()</i> implemented	Gets the Utility Subsystem from the world, requires an object that has implemented <i>GetWorld()</i> .
<i>const UABUtilitySubsystem& GetUtilitySubsystemChecked(const UObject* WorldContextObject)</i>	<i>WorldContextObject</i> - An object with <i>GetWorld()</i> implemented	Gets the Utility Subsystem from the world, requires an object that has implemented <i>GetWorld()</i> . This will crash if the Utility Subsystem cannot be found.
<i>UABUtilitySubsystem* GetMutableUtilitySubsystem(const UObject* WorldContextObject)</i>	<i>WorldContextObject</i> - An object with <i>GetWorld()</i> implemented	Gets the Utility Subsystem from the world, requires an object that has implemented <i>GetWorld()</i> .
<i>UABUtilitySubsystem& GetMutableUtilitySubsystemChecked(const UObject* WorldContextObject)</i>	<i>WorldContextObject</i> - An object with <i>GetWorld()</i> implemented	Gets the Utility Subsystem from the world, requires an object that has implemented <i>GetWorld()</i> . This will crash if the Utility

		Subsystem cannot be found.
<code>template <typename T> const T* GetActionInstanceMemoryOfType(const UABUtilityActionBase& Action, const UABUtilityComponent& Component)</code>	<i>Action</i> - The Action to get the Instance Memory Type for <i>Component</i> - The Utility Component to get the Instance Memory for	Gets the Utility Subsystem and runs <i>GetActionInstanceMemoryForComponent()</i> , passing in <i>Action</i> and <i>Component</i> . Casts to type <i>T</i> and returns the pointer.
<code>template <typename T> T* GetMutableActionInstanceMemoryOfType(const UABUtilityActionBase& Action, const UABUtilityComponent& Component)</code>	<i>Action</i> - The Action to get the Instance Memory Type for <i>Component</i> - The Utility Component to get the Instance Memory for	Gets the Utility Subsystem and runs <i>GetMutableActionInstanceMemoryForComponent()</i> , passing in <i>Action</i> and <i>Component</i> . Casts to type <i>T</i> and returns the pointer.
<code>const FString GetDebugCallstackString(int32 IgnoreCount)</code>	<i>IgnoreCount</i> - The amount of currently stacked functions to ignore in the callstack	Fetches the current callstack, ignoring an amount of calls influenced by <i>IgnoreCount</i> . Implementations of the <i>FPlatformStackWalk::StackWalkAndDump()</i> are platform specific. This is a slow function, try not to have this called on any sort of tick.
<code>void UtilityVLOG(const ELogVerbosity::Type Verbosity, const FString& Message, const UObject* LogOwner)</code>	<i>Verbosity</i> - The level of log <i>Message</i> - The log to print <i>LogOwner</i> - The owning object for the log	Handles logging to the Visual Logger and standard log output. Adds the callstack to the log grabbed from <i>GetDebugCallstackString()</i> if <i>ShouldPrintCallstackInLogs()</i> returns true from the Utility Settings.
<code>EABUtilityActionRunStatus StateTreeRunStatusToUtilityActionRunStatus(EStateTreeRunStatus StateTreeRunStatus)</code>	<i>StateTreeRunStatus</i> - The <i>EStateTreeRunStatus</i> to convert to an <i>EABUtilityActionRunStatus</i>	Converts an <i>EStateTreeRunStatus</i> to an <i>EABUtilityActionRunStatus</i> .

<i>const FVector::FReal CalculateAngleDifference Dot(const FVector& A, const FVector& B)</i>	<i>A</i> - Initial vector <i>B</i> - Secondary vector	Calculates the cosine of the angle between vectors <i>A</i> and <i>B</i> when projected onto a 2D plane.
--	--	--

Examples

Component Setup

The first thing to set up is an AI Controller. Create a custom class that inherits from *AAIController*, expose a pointer to a *UABUtilityComponent* and in the constructor, create the component and set it as the *BrainComponent*.

.h file

```
UCLASS()
class AJIMBOBUTILITYAI_API ATestUtilAIController : public AAIController
{
    GENERATED_BODY()

public:
    ATestUtilAIController(const FObjectInitializer& ObjectInitializer);

private:
    UPROPERTY(EditAnywhere)
    TObjectPtr<UABUtilityComponent> UtilityComponent = nullptr;
};
```

.cpp file

```
ATestUtilAIController::ATestUtilAIController(const FObjectInitializer&
ObjectInitializer)
{
    bStartAILogicOnPossess = true;
    bStopAILogicOnUnposses = true;

    UtilityComponent =
CreateDefaultSubobject<UABUtilityComponent>(TEXT("UtilityComponent"));
    BrainComponent = UtilityComponent;
}
```

Create a Blueprint of this custom AI Controller and, in the *UtilityComponent*, find the *UtilityDataAsset*. This is where the data asset will go once created.

Create a new data asset of type *UABUtilityDataAsset*. Open and save it, and set up the component so that *UtilityDataAsset* is set to this new asset. Now the asset can be built up.

In the component, find the *ActionSelectionMode* dropdown and set this to whichever mode most suits this AI. Now the asset can be filled with Actions.

In the project settings, find *Utility AI* and add the new asset to the *UtilityDataAssets* list. If it is not added here, its Actions will not be available to components at runtime.

Code Action

To create an Action in code, go to the code files and create a new class that inherits from *UABUtilityActionBase*. Name it something that reflects what the Action should make the AI do.

Override *DoAction()*, this is the function where the main logic for the Action will exist.

If the Action requires Ticking, also override *Tick()* as the ticked logic will exist here.

In the *.cpp* file for this class, build up these two functions so that the Action does what is desired. If the Action should have other custom logic, functions such as *PauseAction()*, *ResumeAction()*, and *AbortAction()* can also be overridden to add custom logic to these processes.

If exposed members are being defined in the class, make sure to override

ValidateAction() to ensure that the member is always valid (this will do editor-time validation when the asset is saved). Also override

DescribeSelfToGameplayDebugger() to ensure the value of the member can be shown in the Gameplay Debugger Menu for easier debugging.

Internal members and functions can be added freely to keep the Action code tidy and customisable to users.

Blueprint Action

Another option for creating an Action is to create one in Blueprint. To do this, create a new Blueprint class of type *UABUtilityActionBlueprintBase* and name it something that reflects the purpose of the Action.

In this Action, override the function *DoBlueprintAction()* to define main Action logic, *TickBlueprintAction()* to define any logic that needs to be ticked, and if validation or debugging is desired then the functions *ValidateBlueprintAction()* and *DescribeBlueprintSelfToGameplayDebugger()* can be overridden in the Blueprint to supply these.

Action Setup in Data Asset

In the data asset, add the new Action into the list of Actions. Any custom members will appear under the name of the Action. Base Action settings can be found under the *ABUtility Action Base* section. The most important of these is the *Utility Weight*, as if this is 0 the Action will never run. Set this to a value suitable compared to all Utility Weights for all Actions in this data asset.

The next thing to set up is the Requirements and Factors for the Action.

Code Requirement

To create a Requirement in code, go to the code files and create a new class that inherits from *UABUtilityRequirementBase*. Name it something that reflects what the Requirement is checking.

Override the *CheckRequirement()* function and set it up so that it checks the correct thing. Custom functions and members can be added to ensure code cleanliness and customisability, and members can be validated and debugged in the

ValidateRequirement() and *DescribeSelfToGameplayDebugger()* function overrides respectively.

Blueprint Requirement

Requirements can also be made in Blueprints. Create a new Blueprint class of type *UABUtilityRequirementBlueprintBase* and, in the new Blueprint, override *CheckBlueprintRequirement()* and set up the logic to check for what is required. Custom members can be validated and debugged in the overridable functions *ValidateBlueprintRequirement()* and *DescribeBlueprintSelfToGameplayDebugger()*.

Multi-Requirement

There is an example Requirement that comes with the plugin that is a “Multi-Requirement”. This groups several Requirements together and supplies an operator, either AND or OR. This is a good way to group Requirements and allows for the OR operator if only one of a certain group of Requirements needs to be true for the Action to be possible.

Code Factor

To create a Factor in code, go to the code files and create a new class that inherits from *UABUtilityFactorBase*. Name it something that reflects what the Factor will score against.

Override the *CalculateFactorScore()* function and set it up so it calculates a normalised score. Custom functions and members can be added to ensure code cleanliness and customisability, and members can be validated and debugged in the *ValidateFactor()* and *DescribeSelfToGameplayDebugger()* function overrides respectively.

Blueprint Factor

Factors can also be made in Blueprints. Create a new Blueprint class of type *UABUtilityFactorBlueprintBase* and, in the new Blueprint, override *CalculateBlueprintFactorScore()* and set up the logic that will calculate and return the Factor score. Custom members can be validated and debugged in the overridable functions *ValidateBlueprintFactor()* and *DescribeBlueprintSelfToGameplayDebugger()*.

Requirement and Factor Setup in Data Asset

Back in the data asset, the Action added previously has two lists, one named Requirements and one named Factors. Add the newly created Requirement and Factor into these lists and set up the correct data for any exposed members.

Asynchronous Logic and Parallel For

The plugin comes with built-in asynchronous logic and usage of the engines *AsyncTask* and *ParallelFor* helpers. These are not used automatically, as they may not always help optimise the game due to the added overhead.

To enable the asynchronous logic, go to the component set up in the AI Controller Blueprint and enable *RunComponentAsynchronously*. This will use *AsyncTask* to check the Requirements for each Action before choosing an Action once complete.

To use the *ParallelFor* helper in the component, enable *UseParallelRequirementChecking*. This will swap the generic *for* loop for the *ParallelFor* helper and spread out the iterations across threads.

Each Action also has this option, so it can be enabled for individual Actions if that Action should benefit from it.

On the component, both *RunComponentAsynchronously* and *UseParallelRequirementChecking* can be enabled at the same time, but it is not required.

Ensure to measure performance both before and after enabling or disabling each of these, including after enabling or disabling

***UseParallelRequirementChecking* on each Action individually to make sure the game is running as best as it can.**

Validation

When saving the asset, all of the Actions and their Requirements and Factors will automatically be validated. The *UABUtilityDataAssetValidator* class makes sure it's validating a *UABUtilityDataAsset*, and then runs *ValidateAction()* on each of the data assets Actions, *ValidateRequirement()* on each of each Actions Requirements, and *ValidateFactor()* on each of each Actions Factors. It is wise to override the relevant validation function on each Action, Requirement and Factor to ensure that each member of each class is valid at editor time.

Debugging

The Gameplay Debugger Menu can be enabled at runtime by pressing the apostrophe (') key. One of the categories that should come up will be labelled "UtilityAI". Enabling this category will run *DescribeSelfToGameplayDebugger()* on the Utility Component of the debugged actor and all Actions, Requirements and Factors found within that component, displaying text describing the current values of class members and drawing any debug shapes relating to the current Action being performed. It is wise to override the debugging function on each Action, Requirement and Factor to ensure each member of each class can be debugged at runtime.

Comments

The plugin is built and maintained by Axela Brockett. Any comments regarding improvements, bugs found or anything else can be directed to her either in the GitHub repository comments or on LinkedIn at

<https://www.linkedin.com/in/axelajimbrockett/>.